# Есть ли будущее у генетических и обучаемых методов в оптимизаторах запросов ?

Рыбакина Алена

**PG BootCamp Russia 2026 Moscow**
PGBootCamp.ru

# Структура доклада

**01** **С чего все началось: основная проблема**
Экспоненциальный рост · Left-deep vs Bushy

**02** **Классические алгоритмы**
Selinger · DPhyp · IKKBZ · LinDP++ · A*

**03** **Рандомизированные методы**
GEQO · Simulated Annealing

**04** **ML и Adaptive подходы и их критика**
Neo · Bao · SkinnerDB

**05** **Текущее состояние**
Что работает · Что открыто · Траектория

01

# С чего все началось: проблема

Экспоненциальный рост · Left-deep vs Bushy · DP невозможен

# Проблема 1: слишком много кандидатов

Почему нельзя просто попробовать все — и что с этим делать

| 5 таблиц | 10 таблиц | 15 таблиц | 20 таблиц | 50 таблиц |
|---|---|---|---|---|
| **120** | **3 628 800** | **$1.3 \times 10^{12}$** | **$2.4 \times 10^{18}$** | **$3 \times 10^{64}$** |
| перебор за мс ✓ | займёт секунды | нужны часы ✗ | практически невозможно | вечности не хватит |

# Проблема 1: слишком много кандидатов

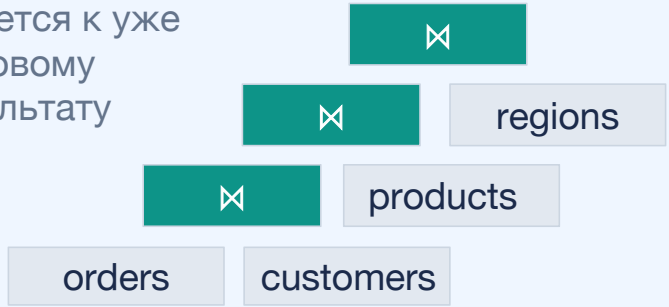
Почему нельзя просто попробовать все — и что с этим делать

| 5 таблиц | 10 таблиц | 15 таблиц | 20 таблиц | 50 таблиц |
|---|---|---|---|---|
| 120 | 3 628 800 | $1.3 \times 10^{12}$ | $2.4 \times 10^{18}$ | $3 \times 10^{64}$ |
| перебор за мс ✓ | займёт секунды | нужны часы ✗ | практически невозможно | вечности не хватит |

## Left-deep план (Postgres-like) - n!

((orders ⋈ customers) ⋈ products) ⋈ regions

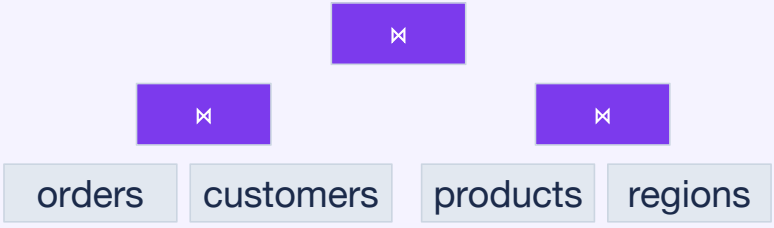
Каждый JOIN добавляется к уже готовому результату




## Bushy план - (2n-2)!/(n-1)!

(orders ⋈ customers) ⋈ (products ⋈ regions)

Оба поддерева строятся параллельно

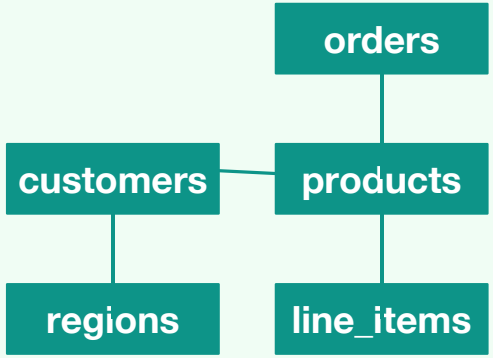


# Все сложнее, если есть цикл

**Ключевое: не важно сколько рёбер - важно есть ли цикл. Одно «лишнее» ребро создающее цикл - и задача из $O(n^2)$ превращается в NP-hard.**

## Дерево

Каждый узел связан ровно с одним «родителем». Есть чёткая иерархия.

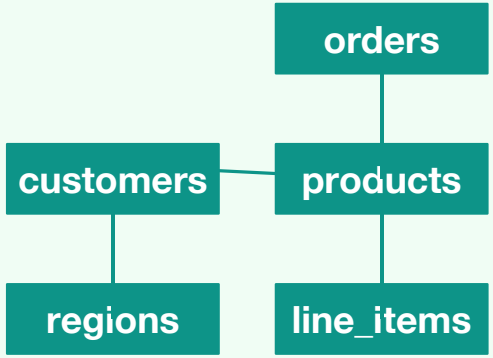


# Все сложнее, если есть цикл

**Ключевое: не важно сколько рёбер - важно есть ли цикл. Одно «лишнее» ребро создающее цикл - и задача из $O(n^2)$ превращается в NP-hard.**

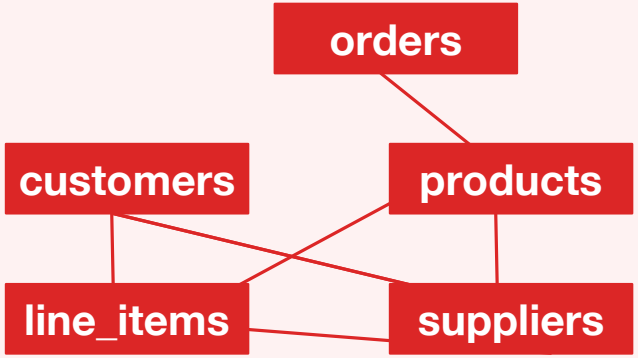
## Дерево

Каждый узел связан ровно с одним «родителем». Есть чёткая иерархия.




## Граф с циклом

Появляется хотя бы один «цикл»:
A → B → C → A.
Нет однозначной иерархии.



# Проблема 2: планировщик всегда ошибается в оценках

*Формула допущения о независимости предикатов: P(A и B вместе) = P(A) × P(B)*

### Город и страна

```
WHERE city = 'Paris'
  AND country = 'France'
```

**Оценка планировщика:**

PostgreSQL считает: 0.001 × 0.05 = 0.00005 (0.005% строк)

**Реальность:**

На самом деле: если город Париж - страна почти всегда Франция. Реальная доля ≈ 0.001.

### Год и десятилетие

```
WHERE year = 2020
  AND decade = '2020s'
```

**Оценка планировщика:**

PostgreSQL считает: 0.01 × 0.1 = 0.001

**Реальность:**

На самом деле: если год=2020, то decade='2020s' гарантированно. Реальная доля = 0.01.

### IMDB: жанр и год фильма

```
movie.year > 2000
  AND keyword.type
  IN ('thriller','drama')
```

**Оценка планировщика:**

PostgreSQL медиана Q-error = 1.0 (половина запросов оценивается точно)

**Реальность:**

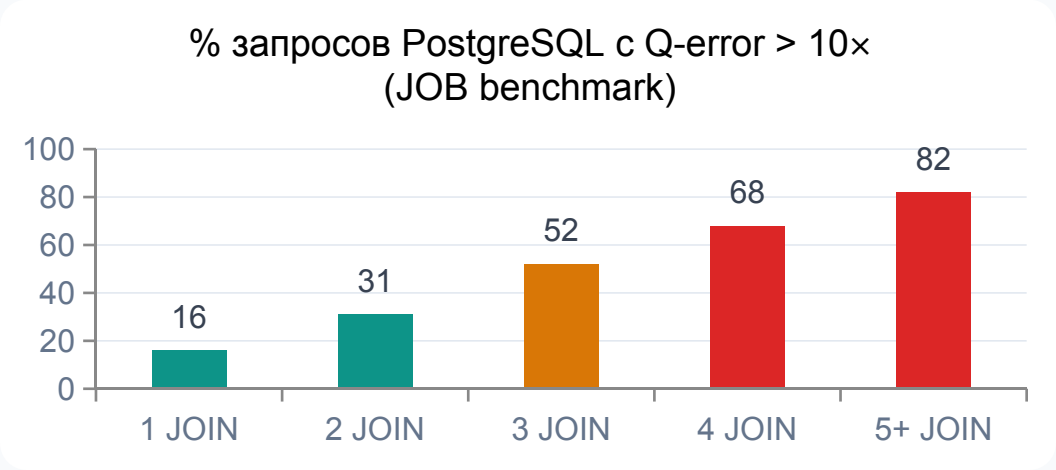
Но 95-й перцентиль Q-error = 6.1 Максимальный Q-error = 207 (!).

*PGCon 2017 (Ivanov): невозможно корректно обработать корреляции только через одномерные гистограммы · Leis VLDB 2015*

# Ошибки оценок умножаются на каждом JOIN

**Каждый JOIN перемножает ошибки предыдущих. Если на каждом шаге ошибка ×2, то на 5 JOIN-ах: $2^5 = 32$×. В реальности хуже - ошибки не просто накапливаются, они умножаются нелинейно.**

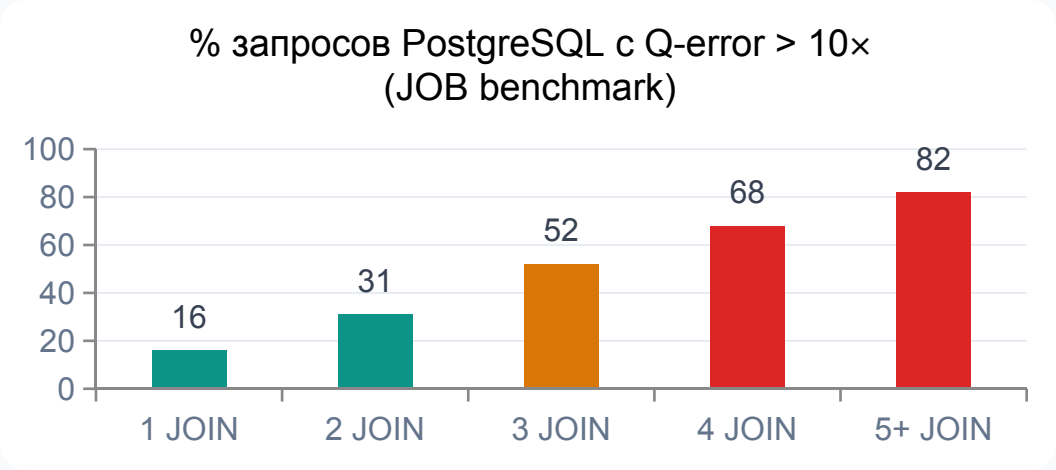
$$q - error = MAX\left(\frac{estimated}{actual}, \frac{actual}{estimated}\right)$$



# Ошибки оценок умножаются на каждом JOIN

**Каждый JOIN перемножает ошибки предыдущих. Если на каждом шаге ошибка ×2, то на 5 JOIN-ах: $2^5 = 32$×. В реальности хуже - ошибки не просто накапливаются, они умножаются нелинейно.**

$$q - error = MAX\left(\frac{estimated}{actual}, \frac{actual}{estimated}\right)$$




**Как нарастает ошибка в одном запросе:**

| JOIN № | Оценивает | На самом деле | Q-error |
|---|---|---|---|
| 1-й | ×2 строки | 1.9 | 1.05× |
| 2-й | ×5 строк | 47 | **9.4×** |
| 3-й | ×12 строк | 890 | **74×** |
| 4-й | ×31 строки | 12 000 | **387×** |
| 5-й | ×78 строк | 180 000 | **2 307×** |

# Предложенное решение

**PostgreSQL хранит статистику по каждой колонке ОТДЕЛЬНО и не знает, как колонки связаны между собой.**

## Расширенная статистика - частичное решение (PG 10+)

Позволяет явно сообщить PostgreSQL о зависимостях между колонками одной таблицы.

**functional_deps**
Позволяет city определяет country.
Теперь P(city='Paris' AND country='FR') = P(city='Paris').

**MCV lists**

Хранит топ-N частых пар (year, genre)

**ndistinct**
Число уникальных КОМБИНАЦИЙ (a,b). Помогает GROUP BY и DISTINCT.

# PGCon 2010–2018: диагностика и первые решения

- PGCon 2010 (Ottawa) · Jan Urbański ·
  «Replacing GEQO with Simulated Annealing (SAIO)»
- PGCon 2011 (Ottawa) · Tom Lane ·
  «Hacking the Query Planner I & II» (2-part tutorial)
- psql-hackers 2011–2012: треды о плохих планах
- PGConf.EU 2015/2016/2017 · Tomas Vondra · «Multivariate Statistics»
- psql-hackers 2016–2017: 2+ года ревью патча расширенной статистики
- PGCon 2017 · Andres Freund · «JIT Compilation for PostgreSQL»

Результат периода: расширенная статистика в PG10 (2017) - первый инструмент для борьбы с independence assumption в PostgreSQL core.
Частичная победа: ndistinct + functional_deps + MCV.

# PGCon 2019–2025: AQO, ML-советники и статистики выражений

**PGCon 2017/2019 · Oleg Ivanov (Postgres Pro) · «Adaptive Query Optimization»**

**psql-hackers 2020–2022: AQO patch review**

**PGConf.Online 2021 · Rouhaud & Torikoshi · «Automatic Performance Adviser»**

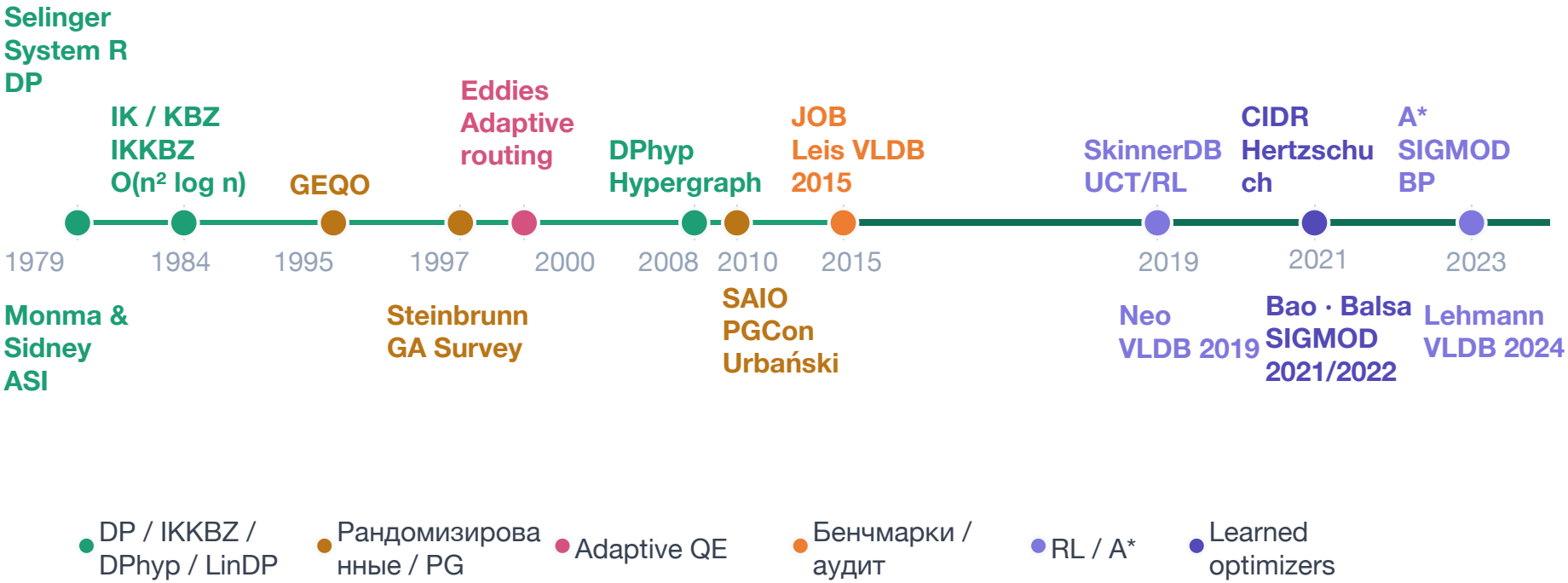
**PGCon 2023 Unconference (Ottawa) · «Cross-table correlations»**

«Можно ли CREATE STATISTICS между таблицами?». «Использование sketches (HyperLogLog, Count-Min) для join selectivity?». «Что если runtime-собирать статистики во время первого выполнения?»

**PG18 (2025): pg_restore_attribute_stats**

**Открытый вопрос 2025: join-crossing correlations**

# 46 лет истории: хронология ключевых работ



# 02

# Классические алгоритмы

Selinger · DPhyp · IKKBZ · LinDP++ · A* · SA/GA

# Selinger DP (1979 г.)

## Идея

Базовая идея: строить план снизу вверх. Сначала найти лучший способ читать каждую таблицу. Потом - лучший способ соединить пары. Потом тройки. И так до полного плана.
Алгоритм хранит оптимальный план для каждого подмножества таблиц и строится снизу вверх за $O(2^n \cdot n)$.

**Ограничение: только left-deep планы**

## Пример с 3 таблицами

k = 1 - одиночные таблицы

| A | B | C |
|---|---|---|
| scan(A) | scan(B) | scan(C) |

k = 2 - все пары (3 варианта)

| {A,B} | {A,C} | {A,C} |
|---|---|---|
| min(A⋈B, B⋈A) | min(A⋈C, C⋈A) | min(A⋈C, C⋈A) |

k = 3 - Полный план: один победитель

{A,B,C}
min(plan{A,B}⋈C, plan{A,C}⋈B, plan{B,C}⋈A)

# Selinger DP (1979 г.)

## Идея

Базовая идея: строить план снизу вверх. Сначала найти лучший способ читать каждую таблицу. Потом - лучший способ соединить пары. Потом тройки. И так до полного плана.
Алгоритм хранит оптимальный план для каждого подмножества таблиц и строится снизу вверх за $O(2^n \cdot n)$.

**Ограничение: только left-deep планы**

## Пример с 3 таблицами

k = 1 - одиночные таблицы

| A | B | C |
|---|---|---|
| scan(A) | scan(B) | scan(C) |

k = 2 - все пары (3 варианта)

| {A,B} | {A,C} | {A,C} |
|---|---|---|
| min(A⋈B, B⋈A) | min(A⋈C, C⋈A) | min(A⋈C, C⋈A) |

k = 3 - Полный план: один победитель

{A,B,C}
min(plan{A,B}⋈C, plan{A,C}⋈B, plan{B,C}⋈A)

# IKKBZ

## Основная идея

Граф запроса - дерево, задача только left-deep JOIN-ordering.
Алгоритм сводит задачу к сортировке отношений по рангу $rank(S) = (T(S) – 1) / C(S)$, где $T(S)$ - размер промежуточного результата, $C(S)$ - стоимость. Выполняется за $O(n^2 \log n)$.
Маленький r → таблица даёт мало строк на единицу стоимости → ставим её раньше, чтобы следующий JOIN работал с меньшим промежуточным результатом.

# IKKBZ

## Основная идея

Граф запроса - дерево, задача только left-deep JOIN-ordering.
Алгоритм сводит задачу к сортировке отношений по рангу rank(S) = (T(S) – 1) / C(S), где T(S) - размер промежуточного результата, C(S) - стоимость. Выполняется за $O(n^2 \log n)$.
Маленький r → таблица даёт мало строк на единицу стоимости → ставим её раньше, чтобы следующий JOIN работал с меньшим промежуточным результатом.

## Пример: цепочка A—B—C—D

У нас четыре таблицы. Orders - центральный узел звезды, к нему присоединяются три других: Customers через предикат p_oc, Products через p_op, Regions через p_or.

**Кардинальности**

Orders: 50 000 строк rank(Orders) ≈ 1.00002
Customers: 200 строк rank(Customers) ≈ 1.005
Products: 500 строк rank(Products) ≈ 1.002
Regions: 5 строк rank(Regions) = 5 / 4 = 1.250

**Сортировка по рангу**

Orders (1.00002) → Products (1.002) → Customers (1.005) → Regions (1.250)

**План**

((Orders ⋈ Products) ⋈ Customers) ⋈ Regions

# DPhyp: гиперграф и connected subgraph pairs

## Идея

**Граф запроса представляется как гиперграф:** узлы - таблицы, гиперрёбра - предикаты JOIN. Алгоритм перебирает только связные пары (csg, cmp) - два непересекающихся подграфа, между которыми есть предикат. Начинаем с одного узла (seed), рекурсивно расширяем его по рёбрам до связного подграфа (csg), затем строим для него дополнение (cmp). Для каждой найденной пары оцениваем стоимость соединения и сохраняем только минимальный план для данного подмножества таблиц.
Никогда не строит декартово произведение.
Сложность $O(3^n)$ worst-case.

# DPhyp: гиперграф и connected subgraph pairs

## Идея

**Граф запроса представляется как гиперграф:** узлы - таблицы, гиперрёбра - предикаты JOIN. Алгоритм перебирает только связные пары (csg, cmp) - два непересекающихся подграфа, между которыми есть предикат. Начинаем с одного узла (seed), рекурсивно расширяем его по рёбрам до связного подграфа (csg), затем строим для него дополнение (cmp). Для каждой найденной пары оцениваем стоимость соединения и сохраняем только минимальный план для данного подмножества таблиц.
Никогда не строит декартово произведение.
Сложность $O(3^n)$ worst-case.

**Преимущества:**
+ Bushy-планы: оба операнда - подзапросы
+ Корректен при циклах в графе запроса
+ Не тратит время на несвязные подмножества

**Где используется:**
✓ DuckDB
✓ Umbra, HyPer
✓ GlareDB
✓ PostgreSQL - только через внешние расширения

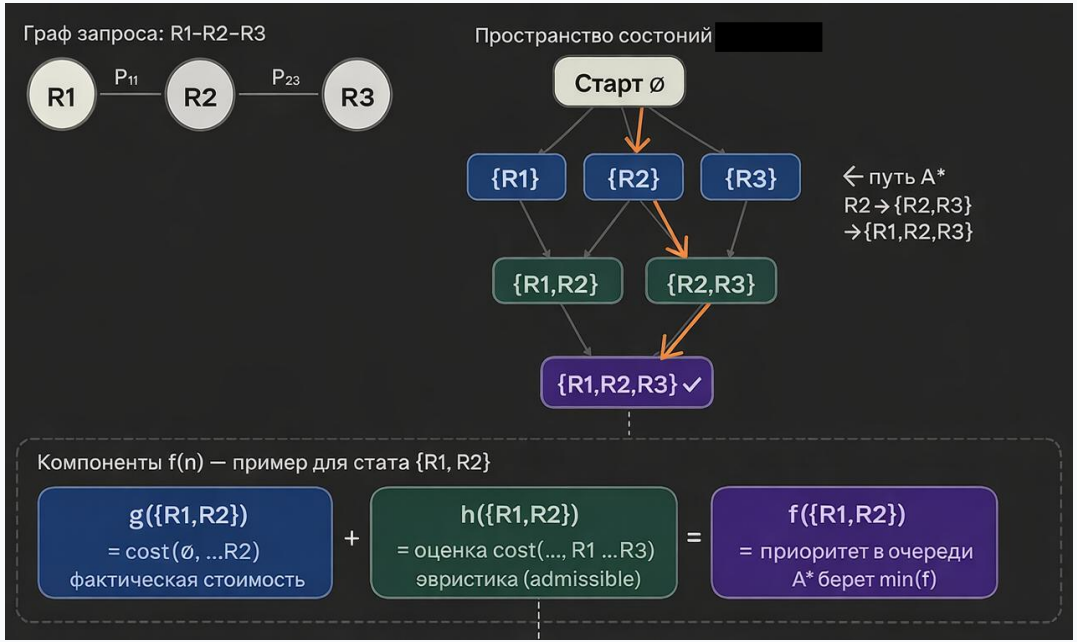
# Сравнение классических алгоритмов

| Алгоритм | Форма плана | Граф запроса | Сложность | Как работает | Где используется |
|---|---|---|---|---|---|
| **Selinger DP (1979)** | Left-deep | Любой | $O(2^n)$ | Оптимальное поддерево строится снизу вверх. Гарантия оптимальности. | PostgreSQL ≤ 12 табл. |
| **IKKBZ (1984/1986)** | Left-deep | Только дерево | $O(n^2 \log n)$ | Вычисляет ранг $r(S)=C(S)/(T(S)-1)$ для каждого поддерева, сортирует по рангу. | теоретический базис |
| **DPhyp (2008)** | Left-deep + Bushy | Любой (гиперграф) | $O(3^n)$ | Гиперграф запроса. Перебирает связные разбиения графа. Без декартовых произведений. | DuckDB, Umbra, HyPer, GlareDB |

# A*

g(n) - стоимость уже выполненных джойнов
h(n) = оценка оставшихся джойнов
f = g+h

1. Начальное состояние ∅ - ни одна таблица не присоединена.
2. Кладём в очередь таблицы.
3. Извлекаем ∅ с минимальным f. Добавляем каждую таблицу по одной.
4. Выбираем дешевую таблицу для сканирования - меньше g.
5. Присоединяем следующие к ней.
6. Лучший результат - JOIN с высокой селективностью, который сильно уменьшает h_sum.
7. и т.д.



# 03

# Рандомизированные методы

GEQO · SA/GA

# PostgreSQL GEQO

**При n > geqo_threshold (по умолчанию 12) exhaustive DP становится неприемлемым по памяти и времени. PostgreSQL переключается на GEQO — генетический оптимизатор.**

## Как работает GEQO:

### Популяция

Создаём N случайных порядков таблиц. Например [3,1,4,2,5...] означает "сначала джойн таблицы 3 с таблицей 1, потом результат с 4" и так далее. Каждый такой порядок - хромосома.

### Оценка

Для каждой хромосомы считаем стоимость плана через стандартный cost model PostgreSQL. Чем меньше — тем лучше.

### Скрещивание

Берём два хороших плана и "смешиваем" их: берём кусок порядка из первого, недостающие таблицы добавляем из второго сохраняя их взаимный порядок. Идея: если в обоих планах таблица A стоит раньше B — это скорее всего неслучайно, надо сохранить.

### Мутация и отбор

Иногда случайно меняем местами две таблицы. Худший план в популяции выбрасываем, лучший новый добавляем. Повторяем несколько поколений - популяция эволюционирует к хорошим планам.

# PostgreSQL GEQO

**При n > geqo_threshold (по умолчанию 12) exhaustive DP становится неприемлемым по памяти и времени. PostgreSQL переключается на GEQO — генетический оптимизатор.**

## Как работает GEQO:

**Популяция** | **Оценка** | **Скрещивание** | **Мутация и отбор**

## Проблемы GEQO:

### Популяция случайная → недетерминированность

Начинаем со случайных перестановок, значит два одинаковых запроса могут получить разные планы. Без фиксации geqo_seed поведение непредсказуемо.

### Оценка считает только total_cost → нет startup_cost

Для LIMIT 10 важно получить первые строки быстро, а не минимизировать суммарную стоимость. GEQO этого не видит.

### Скрещивание субоптимальное

При скрещивании потомок наследует соседства таблиц из обоих родителей. Но в джойнах стоимость соединения зависит от контекста - одни и те же две таблицы рядом могут стоить по-разному. Поэтому унаследованные "хорошие соседства" в новом плане могут оказаться бесполезными.

# Simulated Annealing

## Стратегия: move

**Что делает:**

Выбрать случайный поддерево. Переместить его в другую позицию дерева плана.

**Зачем:**

Позволяет глобально изменять план без полного пересчёта. Самая эффективная операция для escape из локального минимума.

## Стратегия: pivot

**Что делает:**

Выбрать ребро в дереве. Переставить два поддерева, связанных этим ребром (повернуть вокруг ребра).

$(A \bowtie B) \bowtie C \rightarrow A \bowtie (B \bowtie C)$.

**Зачем:**

Быстрая локальная операция. Хороша для fine-tuning внутри найденного хорошего региона.

## Стратегия: recalc

**Что делает:**

Полный пересчёт стоимости плана явно, не инкрементально. Происходит периодически.

**Зачем:**

Компромисс точность/скорость. Позволяет делать больше итераций за то же время.

# SAIO: Simulated Annealing (PGCon 2010)

**PGCon 2010 (Ottawa) · Jan Urbański · «Replacing GEQO with Simulated Annealing (SAIO)»**

| Старт | Делаем ход | Принимаем или нет | Равновесие | Заморозка |
|---|---|---|---|---|
| Строим начальный план left-deep. Устанавливаем температуру T = I × число_таблиц | Берём два случайных узла в дереве плана и меняем их поддеревья местами. Пересчитываем стоимость только затронутой части дерева - от переставленных узлов вверх до их общего предка. | Новый план дешевле → берём всегда<br>Новый план дороже → берём с вероятностью e^((старая_стоимость – новая_стоимость) / T) | Сделали N × число_таблиц шагов на этом уровне температуры и далее снижаем температуру: T = T × K (например K = 0.6) | Если T < 1 и несколько шагов подряд не принято - останавливаемся.<br><br>Возвращаем лучший найденный план. |

# SAIO: Simulated Annealing (PGCon 2010)

**PGCon 2010 (Ottawa) · Jan Urbański · «Replacing GEQO with Simulated Annealing (SAIO)»**

| Старт | Делаем ход | Принимаем или нет | Равновесие | Заморозка |
|---|---|---|---|---|
| Строим начальный план left-deep. Устанавливаем температуру T = I × число_таблиц | Берём два случайных узла в дереве плана и меняем их поддеревья местами. Пересчитываем стоимость только затронутой части дерева - от переставленных узлов вверх до их общего предка. | Новый план дешевле → берём всегда<br>Новый план дороже → берём с вероятностью e^((старая_стоимость – новая_стоимость) / T) | Сделали N × число_таблиц шагов на этом уровне температуры и далее снижаем температуру: T = T × K (например K = 0.6) | Если T < 1 и несколько шагов подряд не принято - останавливаемся.<br><br>Возвращаем лучший найденный план. |

## Технические проблемы

Код делал грязные хаки с внутренностями планировщика

Утечки памяти между шагами при неаккуратной реализации

## Результаты неоднозначные

На среднем запросе GEQO даёт лучший план.
На большом - SAIO в 5 раз быстрее.

## Организационные причины

Параметры K и N нужно подбирать вручную
GEQO уже работает и достаточно хорош для большинства запросов

# Сравнение алгоритмов: итоговая таблица

| Алгоритм | Сложность | Optimal | При 100 таблиц | Производство |
|---|---|---|---|---|
| Selinger DP | $O(2^n)$ | ✓ left-deep | ✗ | PostgreSQL (n≤12), MySQL, Oracle |
| DPhyp | O(csg) | ✓ | clique: до 14<br>✓ для остальных | DuckDB, Umbra, HyPer, GlareDB |
| IKKBZ | $O(n^2)$ | ✓ left-deep | ✓ | CockroachDB |
| A* | worst-case $O(2^n)$<br>**на практике sub-exp | ✓ | ✓ | Research only (SIGMOD 2023) |
| GEQO (GA) | O(k·n) | ✗ | ✓* | PostgreSQL (n>12) |
| SAIO (SA) | O(k·n) | ✗ | ✓* | Research прототип |

*работает за разумное время, но не даёт оптимальный план

**sub-exp — хуже полинома, но лучше экспоненты

04

# ML и Adaptive подходы

Neo · Bao · SkinnerDB · AQO — ожидания vs реальность

# Ландшафт подходов к оптимизации JOIN-ов

| SkinnerDB Online RL | AQO k-NN hist. | Neo Tree-CNN | Bao Hint+Bandit |
|---|---|---|---|
| *SIGMOD 2019* | *PGCon 2017* | *VLDB 2019* | *SIGMOD 2021* |

**Сравнение по ключевым измерениям:**

| Подход | Гарантия оптим-ти | Масштаб таблиц | CE-ошибки | Без статистик | Production |
|---|---|---|---|---|---|
| SkinnerDB (RL) | Regret bound | Любой | Не нужны | ✓ | Research прототип |
| Neo / Bao | ✗ | ≤ ~30 | Частично | Нет | Research |
| BAO | Bounded | <=100 | Частично | Нет | Research |
| AQO (PG) | ✗ | Любой | улучшает CE | Нет | Postgres Pro ext. |

Regret bound - гарантия что отставание от оптимального плана ограничено сверху и не растёт бесконечно.
Bounded - со временем сходится к хорошим планам, но без строгой математической гарантии.
CE-ошибки - оптимизатор неверно предсказывает размер промежуточных результатов, из-за чего выбирает плохой план.

# Neo (VLDB 2019)

**Идея: нейросеть сама учится предсказывать реальное время выполнения плана. Тогда можно перебрать несколько вариантов, предсказать время каждого и выбрать лучший - без модели стоимости.**

## Как работает Neo:

| SQL Query | Feature Encoding | Tree-CNN | Latency Predictor | Greedy Plan Build |
|---|---|---|---|---|
| Получаем SQL-запрос | Какие предикаты активны, какие таблицы | Кодируем дерево плана в вектор | Предсказываем реальное время в мс | Жадно добавляем таблицы |
| дерево физического плана выполнения запроса | - Тип оператора<br>- Статистика таблицы/предиката<br>- Параметры оператора | Для каждого внутреннего узла применяется свёрточный слой который объединяет вектор узла с векторами его потомков. | Принимает: вектор плана из Tree-CNN + опционально вектор "контекста запроса" (таблицы, предикаты, GROUP BY, ORDER BY). Выдаёт: предсказание в мс. | - Перечисление кандидатов для каждой пары таблиц<br>- Каждый частичный план кодируется Tree-CNN и оценивается latency predictor-ом<br>- Фиксация и повтор |

# Neo (VLDB 2019)

## Как работает Neo:

| SQL Query | Feature Vector | Tree-CNN | Latency Predictor | Greedy Plan Build |
|---|---|---|---|---|
| Получаем SQL-запрос | Какие предикаты активны, какие таблицы | Кодируем дерево плана в вектор | Предсказываем реальное время в мс | Жадно добавляем таблицы |

**Результаты на JOB (113 запросов IMDB):**

- медиана: +18% - типичные запросы чуть лучше
- 95-й перцентиль: +29% - медленные запросы улучшились сильнее
- максимум: +32% - самый медленный запрос улучшился больше всего

**Почему не production-ready:**

Использует планы PostgreSQL как обучающие данные - наследует его систематические ошибки.

Нет защиты от регрессий: Если нагрузка изменилась - Neo может выбрать в 100× худший план.

2+ часа переобучения: При каждом изменении схемы или данных — нужно переучивать. Неприемлемо.

# Bao (SIGMOD 2021)

**Bao не строит план сам - он передаёт PostgreSQL «подсказки» (hints) и смотрит что получится.**
**Worst case = стандартный PostgreSQL. Регрессий хуже baseline нет.**

**Что такое hint sets — и как Bao их использует:**

| | | | | |
|---|---|---|---|---|
| Enable Hash Join | Enable Nested Loop | Enable Index Scan | Enable Merge Join | Force Parallel |

~12 таких hint sets. Выбирает hint с наибольшей неопределённостью → передаёт в PG → получает реальное время → обновляет модель.

**SQL Query** – **PG optimizer + hint** – **Execute → real time** – **Update Tree-CNN** – **Лучший hint в следующий раз**

**JOB benchmark:**

Улучшение vs PostgreSQL: +38% на JOB, +10% на Neo. При этом НИКОГДА не хуже PostgreSQL - bounded regret гарантирован.

На STACK benchmark (2600 запросов, сложнее JOB):
inference overhead > выигрыш для коротких запросов.
Требует внешнего ML-сервиса.

# Критика подходов

**Вывод Lehmann, Sulimov, Stockinger 2024: «PostgreSQL превосходит все LQO при честном тестировании. Прогресс последних 5 лет - методологический артефакт, а не реальное улучшение».**

## 01 Их тренировали и тестировали на одних данных

Авторы LQO обучают модель на 80% запросов JOB и проверяют на оставших 20% - из того же набора. Нейросеть просто запоминает конкретные запросы, а не учится оптимизировать. При честном разделении PostgreSQL выигрывает.

## 02 Не считали время работы самой нейросети

Не учитывался Inference нейросети - 10–50 мс. Итого: нейросеть тратит столько же или больше, чем выигрывает. Авторы LQO считали только время SQL-выполнения, игнорируя время планирования.

## 03 Первый запуск всегда медленнее

Первое выполнение любого запроса на 14.6% медленнее второго - из-за холодного кэша операционной системы (cold I/O cache). Случайный выбор: если LQO делает warm run, PG делает cold — LQO выигрывает. И наоборот.

## 04 JOB benchmark слишком мал

JOB — 113 запросов на базе IMDB. Слишком мало для надёжного ML. STACK benchmark — 2600 запросов StackExchange. На STACK: Neo и Bao дают timeouts на сложных запросах. PostgreSQL на STACK существенно стабильнее.

# SkinnerDB (SIGMOD 2019)

**Радикальная идея: выполняем запрос, пробуя разные порядки JOIN на ходу и учимся на результатах. Гарантия: суммарное отставание от оптимального плана растёт не быстрее O(√T* · log T*). Чем дольше работает - тем ближе к оптимуму.**

## Как это работает:

| Шаг 1 - Выбор следующего джойна | Шаг 2 - Выполняем короткий слот | Шаг 3 - Обновляем статистику | Шаг 4 - Повторяем | Конец |
|---|---|---|---|---|
| Для каждого возможного следующего джойна считаем UCB-score: score = avg_reward+C* sqrt(ln(total_visits) / visits) Выбираем джойн с максимальным score. | Выполняем выбранный порядок джойнов несколько миллисекунд. Считаем сколько новых строк результата получено — это и есть награда. | Обновляем avg_reward и visits для этого порядка джойнов. Порядки которые быстро дают строки — получают выше score и чаще выбираются в следующих раундах. | Возвращаемся к шагу 1. Постепенно система сама находит быстрые порядки джойнов — без планировщика. | Когда все строки результата получены — запрос завершён. Возвращаем результат. |

T* - это количество временных слотов, которые система уже выполнила.

05

# Текущее состояние

Что работает · Что открыто · Траектория

# Текущее состояние: что есть, что нет, что будет

## ✓ Что используем прямо сейчас

- SelingerDP, GEQO, DHyp
- Рандомизированные алгоритмы (GEQO, SAIO)
- SkinnerDB - единственный с доказанным regret bound.
- Extended STATISTICS
- Adaptive Query Execution: Spark 3.0+ перепланирование на shuffle
- AQO extension
- DP с эвристиками и прунингом

## ✗ Не решено, но что-то в процессе

- Join статистика
- Автоматическое создание расширенной статистики
- Нет стандарта для сравнения LQO
- Применение MCTS и его гибриды для join ordering

| 46 лет | 20+ | NP-hard | $\leq O(n^2)$ |
|---|---|---|---|
| истории | ключевых работ | в общем случае | лучший алгоритм |

# Спасибо! Вопросы?

Рыбакина Алена

VK: https://vk.com/alena_rybakina
LinkedIn: https://www.linkedin.com/in/alena-rybakina
email: alenka.rybakina@gmail.com

PG BootCamp Russia 2026 Moscow
PGBootCamp.ru